Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Hashing and Encryption (in C#)

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Hashing in .NET
- Symmetric Encryption (AES) in .NET
- Asymmetric Encryption (RSA) in .NET
- Other .NET Cryptography APIs

UNIVERSITY of HULL

Cryptographic Recap

- We've looked at Hashing, Symmetric Cryptography and Asymmetric Cryptography
- We've identified how these principles are useful and how they are used.
- If we were to want to employ this type of cryptography we should **not** recreate the algorithms ourselves unless we have no other choice!
- Thankfully they have been designed, created, tested and developed for us already as part of the .NET framework and are available using pre-existing libraries.
- We should, however, consider the fact that we must place all of our trust in these libraries.
- It is important that they are open-source so everyone can inspect them for vulnerabilities!

UNIVERSITY of HULL

Cryptography API in .NET

Provided by `System.Security.Cryptography`

Crypto service providers
Encapsulate the cryptographic routines

Hash functions, e.g.

- MD5CryptoServiceProvider
- SHA1CryptoServiceProvider
- SHA256CryptoServiceProvider

Note: MD5 is now considered ‘cryptographically broken’ and is no longer secure for password storage
- It can still be used for checksums but only against **unintentional** corruption

Note: SHA1 is also cryptographically “poor” as it is breakable with enough power
- see: https://shattered.io/

UNIVERSITY of HULL

Hashing API in .NET

- SHA1 hash function is managed by `System.Security.Cryptography.SHA1`

– Input data is in the byte[] form

– Hash is also in the byte[] form

– 20 bytes long

Usually converted to a hexadecimal string of length 40

```
string Message = "Hello!";
byte[] asciiByteMessage = System.Text.Encoding.ASCII.GetBytes(Message);
byte[] sha1ByteMessage;
using (SHA1 sha1Provider = SHA1.Create())
{ sha1ByteMessage = sha1Provider.ComputeHash(asciiByteMessage); }
```

UNIVERSITY of HULL

Hashing API in .NET

- SHA256 hash function is managed by `System.Security.Cryptography.SHA256CryptoServiceProvider`
  - Input data is in the byte[] form
  - Hash is also in the byte[] form
  - 32 bytes long

Usually converted to a hexadecimal string of length 64

```
string Message = "Hello!";
byte[] asciiByteMessage = System.Text.Encoding.ASCII.GetBytes(Message);
byte[] sha256ByteMessage;
using (SHA256 sha1Provider = SHA256.Create())
{ sha256ByteMessage = sha256Provider.ComputeHash(asciiByteMessage); }
```

UNIVERSITY of HULL

Hashing API in .NET

Conversion of a Byte Array (`byte[]`) to a Hexadecimal String (normally how hashes are represented)

```
public static string ByteArrayToString(byte[] ba)
{
    StringBuilder hex = new StringBuilder(ba.Length * 2);
    foreach (byte b in ba)
        hex.AppendFormat("{0:x2}", b);
    return hex.ToString();
}
```

- Simpler to use the BitConverter (which does the above for you):

```
string hexOutput = BitConverter.ToString(hashAsByteArray);
```

Note that the BitConverter inserts delimiter character '-', e.g:

Hello in SHA1 is: 69-34-2C-5C-39-E5-AE-5F-00-77-AE-CC-32-C0-F8-18-11-FB-81-93

Or: 69342C5C39E5AE5F0077AECC32C0F81811FB8193

UNIVERSITY of HULL

Hashing API in .NET

Doing the Reverse: Conversion of a Hexadecimal String to a Byte Array

Hex strings should always be an even number of characters

There are 2 characters per one byte in hex

```
public static byte[] StringToByteArray(String hex)
{
    int NumberChars = hex.Length;
    byte[] bytes = new byte[NumberChars / 2];
    for (int i = 0; i < NumberChars; i += 2)
        bytes[i / 2] = Convert.ToByte(hex.Substring(i, 2), 16);
    return bytes;
}
```

Or (equivalent):

```
byte[] bytes = Convert.FromHexString(hex);
```

Remember to strip out delimiters if you have used them!

UNIVERSITY of HULL

Symmetric Cryptography API in .NET

Still provided by `System.Security.Cryptography`

Symmetric Cryptography Function Examples:

DES:	DES.Create()
TripleDES:	TripleDES.Create()
AES:	Aes.Create()
RC2:	RC2.Create()

Note:

DES and RC2 are no longer considered cryptographically secure.

TripleDES (sometimes written 3DES) performs DES three times to increase security. It is still considered secure enough but professional advice is that it won't be for long.

AES is considered very strong (especially the default 256-bit variant)

UNIVERSITY of HULL

Symmetric Cryptography API in .NET

Remember block chaining as an important technique for hashing and encryption?

- Output from conversion of previous block is an input into the next block
- This means that all blocks are reliant on the correct decryption of all previous blocks
- And any patterns which give away the plaintext are hidden in the ciphertext

How does this work for the first block?

- An initialisation vector is used (IV)
- This is fed in in place of a previous output
- The IV doesn't have to be kept secret as long as the key is.

As the IV is randomised it helps secure the first block of ciphertext

Without an IV the first block constitutes an attack vector because plaintext patterns can still exist in the ciphertext

However, you should **change the IV each time you use the same key**
otherwise the ciphertext is undermined gradually the more times you use the same IV.

✓ As it can be sent in plaintext this isn't a problem

UNIVERSITY of HULL

Encrypting to a Byte Array Using AES in .NET

```
string message = "hello";
byte[] encryptedMessageBytes;

using(Aes aesProvider = Aes.Create())
{
   aesProvider.GenerateKey(); // creation of shared key
   aesProvider.GenerateIV(); // creation of shared initialisation vector
   ICryptoTransform encryptor = aesProvider.CreateEncryptor();

   using (MemoryStream ms = new MemoryStream())
   {
       using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
       {
       using (StreamWriter sw = new StreamWriter(cs))
           {
               sw.Write(message);
           }
           encryptedMessageBytes = ms.ToArray();
       }
   }
}
```

Remember that AES is a single key (symmetric) algorithm so <u>both sides</u> can encrypt *and* decrypt

You'll need to store the KEY and IV

UNIVERSITY of HULL

Decrypting From a Byte Array Using AES in .NET

```
byte[] encryptedMessageBytes;
string plaintext;

using(Aes aesProvider = Aes.Create())
{
  aesProvider.IV = ivByteArray; // shared IV
  aesProvider.Key = keyByteArray; // shared Key
   ICryptoTransform decryptor = aesProvider.CreateDecryptor();
   using (MemoryStream ms = new MemoryStream(encryptedMessageBytes))
   {
       using (CryptoStream cs = new CryptoStream(ms, decryptor, CryptoStreamMode.Read))
       {
           using (StreamReader sr = new StreamReader(cs))
           {
               plaintext = sr.ReadToEnd();
           }
       }
   }
}
```

Remember that AES is a single key (symmetric) algorithm so <u>both sides</u> can encrypt *and* decrypt

You'll need to set the KEY and IV in a new aesProvider before decryption

UNIVERSITY of HULL

Asymmetric Cryptography API in .NET

Still provided by `System.Security.Cryptography`

Symmetric Cryptography Function Examples:

RSA: RSA.Create()

ECDiffieHellman: ECDiffieHellman.Create()

Note:

RSA can be used for Encryption/Decryption/Key Exchange/Signing

ECDiffieHellman can only be used for Key Exchange and Signing

Both use Public/Private key pairs

UNIVERSITY of HULL

Asymmetric Cryptography – Storing the Private Key

Theoretically, we want to ensure that the private key is stored securely, and only available to our application.

It is useful if a private key is stored in persistent memory (e.g. a hard drive) so that we can retrieve the same key after the application restarts.

However, this risks other applications accessing our private key (which is, of course, unsafe).

Unfortunately there isn't a perfect solution to this.

- On Windows, a MachineKeyStore exists which keeps our keys locked at OS level – it only gives out the keys to authorised applications.
- On iOS, a KeyChain exists – but currently there is no way to read/write to this directly from .NET
- On Linux … some distributions have some kind of key store but most do not. It's not standardised so there is no realistic way for .NET to support it.
- Therefore, it's often left to the developer if you want your code to be cross-platform.
- Often, the modern solution is to use a Cloud Key Vault/Key Management Service

UNIVERSITY of HULL

Encrypting to a Byte Array Using RSA in .NET (legacy)

```
string message = "hello";
byte[] asciiByteMessage = System.Text.Encoding.ASCII.GetBytes(Message);

CspParameters cspParams = new CspParameters(); // CryptoServiceProvider Parameters
cspParams.Flags = CspProviderFlags.UseMachineKeyStore; // Windows only
using(RSA rsaProvider = new RSACryptoServiceProvider(cspParams))
{ byte[] encryptedBytes = rsaProvider.Encrypt(asciiByteMessage, RSAEncryptionPadding.OaepSHA1); }
```

You don't HAVE to use CspParameters (CryptoSeviceProviderParameters) but it's a good idea to use it so you can ensure your application is using the MachineKeyStore.

The MachineKeyStore is an isolated area on the Windows platform that stores and handles cryptographic keys.

It prevents the key from ever being in your application memory, or allows you to minimise the time it is – reducing the chances that malicious code will get access to your keys!

UNIVERSITY of HULL

Encrypting to a Byte Array Using RSA in ASP.NET (newer)

- Use the inbuilt Data Protection service:

```
using Microsoft.AspNetCore.DataProtection;

builder.Services
    .AddDataProtection()
    .SetApplicationName("MyApp")
    .PersistKeysToFileSystem(new DirectoryInfo(@"C:\ProgramData\MyApp\Keys"))
    .ProtectKeysWithDpapi(); // Uses the Windows Key Ring
```

Then make a custom service that:

- Generates a new key on first run
- Converts the key to bytes
- Uses the data protector to protect (encrypt and manage access to) the bytes
- Gets the key from the data protector on subsequent runs instead of making a new one

UNIVERSITY of HULL

Why you might want code isolation:

If we manage keys in the logic server then

- A server compromise means that the key can be compromised and this could cause permanent damage.
- When we want to change keys we may need to redeploy.

If we manage keys in a separate server (e.g. a vault or Key Management Service) then

- The key isn't exposed when the logic server is compromised
- Damage can be stopped by just revoking access to (or switching off) the KMS
- The KMS can regenerate or change keys without changing the logic server

A KMS is safe (enough) for most applications because:

- You trust your KMS
- You have a secure, authenticated connection to it (e.g. TLS – HTTPS)
- It uses authorisation so is only going to talk to you (strong access control)

UNIVERSITY of HULL

Encrypting to a Byte Array Using RSA in .NET **(Universal)**

```
string message = "hello";
byte[] asciiByteMessage = System.Text.Encoding.ASCII.GetBytes(Message);

using(RSA rsaProvider = new RSACreate())
{ byte[] encryptedBytes = rsaProvider.Encrypt(asciiByteMessage, RSAEncryptionPadding.OaepSHA1); }
```

There are a few padding options as well as OaepSHA1

We're focusing on this one because it works for both the (older) Windows-only solution and the (newer) universal solution.

With the universal solution you'll need to work out how to store your own private key (maybe an encrypted file/database?) but because it's newer you get access to some of the more secure padding modes such as OaepSHA256.

UNIVERSITY of HULL

Decrypting From a Byte Array Using RSA in .NET

```
byte[] decryptedBytes = rsaProvider.Decrypt(encryptedBytes, RSAEncryptionPadding.OaepSHA1);
string plaintext = Encoding.ASCII.GetString(decryptedBytes);
```

- Of course… it's sadly not quite that simple.
  - The code samples so far assume the use of the same RSA provider. However, the client needs to be configured to use the server's public key.
  - This means the server formatting it nicely and sending it over to the client (**false:** public key only!):

```
string publicKey = rsaProvider.ToXmlString(false);
```

  - And the client using it to create a new RSA object before encryption:

```
RSA clientRsaProvider = RSA.Create();
clientRsaProvider.FromXmlString(publicKey);
```

UNIVERSITY of HULL

Remember…

It is still possible for a MITM attacker to attack the connection and replace the public key

So we still need a mechanism for trust, using a trusted 3rd party…

… and a protocol to manage this.

Secure Sockets Layer (SSL) and Transport Layer Security (TLS)

TLS does all of the Asymmetric Key Exchange, Symmetric Key Generation and Certificate Signing for you for the HTTP protocol!

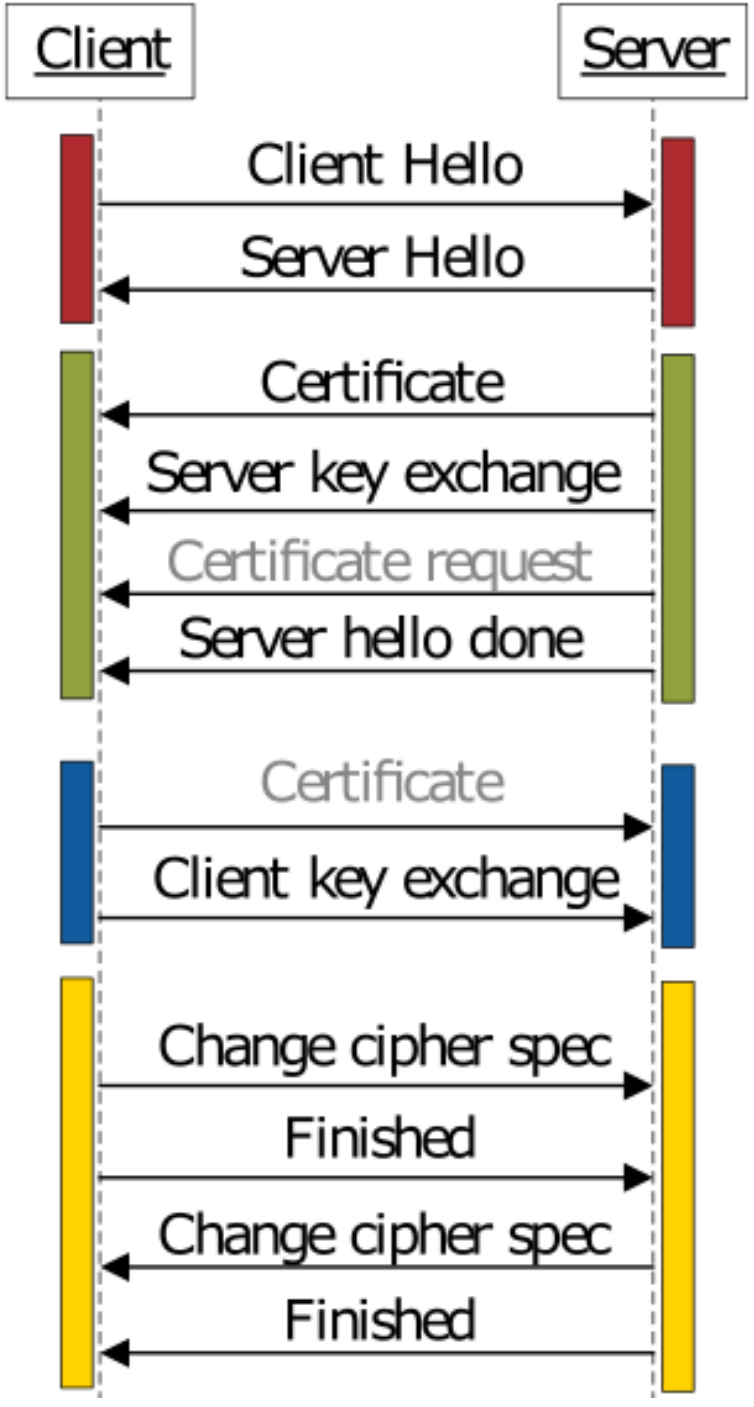
- Secure transport layer for communications on TCP
- SSL is a less secure predecessor of TLS
- Acts as a hybrid cryptosystem
- Services provided by TLS
  - *Server authentication*: asymmetric cryptography
  - *Confidentiality*: symmetric encryption
  - *Integrity*: signature algorithm
  - Exchange of keys for symmetric encryption

TLS is what your browser is using if it shows HTTPS

UNIVERSITY of HULL

Transport Layer Security (TLS)

- Phase 1: Establishing security capabilities
  - Client Hello, Server Hello
  - Client offers TLS version, ciphers, compression
  - Server chooses TLS version, ciphers, compression
- Phase 2: Server authentication and key exchange
  - Certificate, Server key exchange, Certificate request, Server hello done
    - Server sends certificate and key exchange parameters
    - Optional (but recommended): client verifies server certificate
  - Optional: server may request client certificate
- Phase 3: Client authentication and key exchange
  - Certificate, Client key exchange
  - Optional: client sends certificate & server verifies
  - Client sends key exchange parameters
- Phase 4: Finish
  - Change cipher spec, Finished,
  - Confirmation of cipher suites, establishing of encrypted channel



Enabling HTTPS in ASP

ASP comes with some inbuilt HTTPS logic.

Whereas HTTP traffic usually uses port 80, HTTPS usually uses port 443

This will require a certificate to be generated, usually through a trusted Certificate Authority (or locally for testing)

Option 1: deny HTTP requests in the host environment

- This prevents any client from getting through to your application using HTTP
- Although it is not possible to stop a client trying – so they might send insecure private data over HTTP even though it is closed

Option 2: Redirect to HTTPS **(DEFAULT)**

- This prevents your server from responding using HTTP
- Although it is not possible to stop a client trying – so *they* might send insecure private data over HTTP

Option 3: check the Request Uri Scheme in the pipeline, using a custom attribute, inheriting from ActionFilterAttribute

- if(request.RequestUri.Scheme != Uri.UriSchemeHttps) – return (for example) a ‘forbidden’ response
- This allows you to decorate controllers to prevent HTTP requests from accessing certain functionality
- Or redirect via HTTPS

UNIVERSITY of HULL

Back to Hashes: Rainbow Tables and Dictionary Lookups

If hashes are one-way, is it impossible for someone to find out your password once hashed?

Huge amounts of computer resource time have been put into hashing every possible permutation of chars for ever increasing plaintext lengths to ‘break’ passwords stored as hashes.

- SHA1, lowercase and numeric has been completely broken for passwords of 1-10 characters
- There are many terabytes of dictionary lookup ‘rainbow tables’ dedicated to these broken hashes freely available: e.g. http://project-rainbowcrack.com/table.htm

Some cracking attempts are more sophisticated, looking for common words and terms, substitutions and additions (e.g. H£11oW0r7d) to create ‘dictionary attacks’ which provide lookups for likely passwords

- This is estimated to be sufficient to cover as many as 98% of the passwords used on the internet

So what can we do that doesn’t require regularly creating new hash functions?

UNIVERSITY of HULL

Salt

Salting is a process of the addition of some random data into a cryptographic function.

- It is almost exclusively used for hashing.

- When generating the hash of a password, a random salt is created and appended to the password before hashing.
- <u>The salt is stored alongside the hash in the database.</u>
- When comparing the hash of a password, the salt data is appended to the password.

Therefore, a terrible password that is already cracked:

'Password123456'

is not going to show up in your database as the cracked hash

It would take an attacker a lot of time/power to apply the random salt to each and every possible password

Salting makes hashes **<u>MUCH</u>** stronger but is made even better with pepper…

UNIVERSITY of HULL

Pepper

Pepper is also a process of the addition of some random data into a cryptographic function.

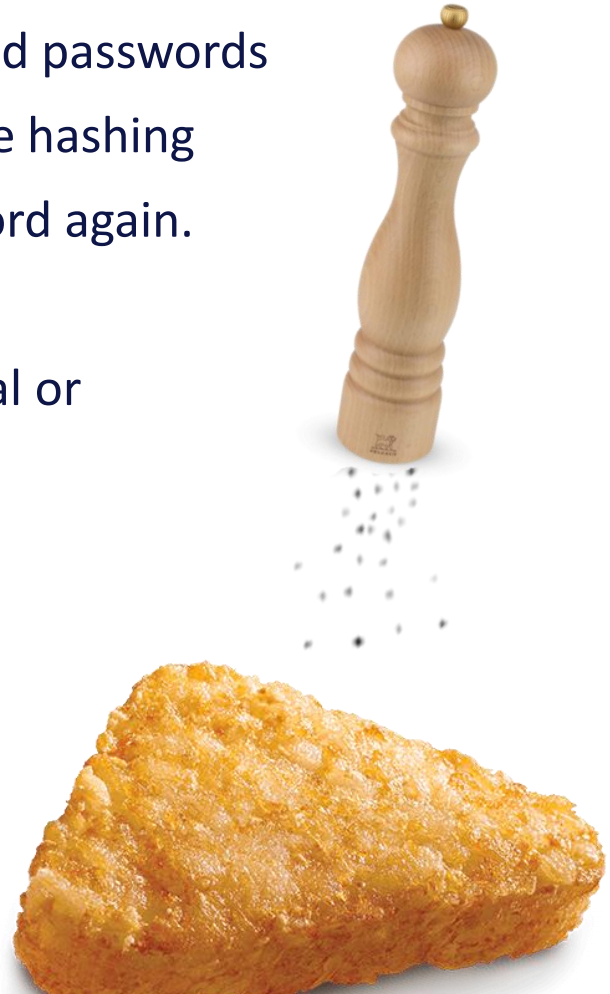
- It is almost exclusively used for hashing.

- The server keeps a single <u>secret</u> “pepper” code that is not stored alongside the hashed passwords
- When generating the hash of a password, pepper is appended to the password before hashing
- When comparing the hash of a password, the pepper data is appended to the password again.

- If someone stole your database full of hashed passwords they would also have to steal or <u>crack your pepper</u>.

- **<u>The most secure type of hash is a salt and pepper hash.</u>**

- Yes – really, I’m not making this up!



Find out if I'm making this up…

- **Further Reading**

- Jonathan LeBlanc, Tim Messerschmidt, *Identity and Data Security for Web Development: Best Practices*, Oreilly

- Wenz, Christian. ASP.NET Core Security. Manning
- Tutorials on MSDN and other web resources
  - System.Security.Cryptography docs: https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography
  - .NET Cryptography Model: https://docs.microsoft.com/en-us/dotnet/standard/security/cryptography-model
  - ASP.NET Security Overview: https://docs.microsoft.com/en-us/troubleshoot/aspnet/security-overview
  - What is TLS/SSL?: https://www.acunetix.com/blog/articles/tls-security-what-is-tls-ssl-part-1/

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Hashing in .NET
- Symmetric Encryption (AES) in .NET
- Asymmetric Encryption (RSA) in .NET
- Other .NET Framework Cryptography APIs
- TLS/SSL
- Salting
- Peppering

In the next session we will be looking at Concurrency

UNIVERSITY of HULL